

IN3030 – Oblig 2

The processor used in this oblig was the M1 chip which consists of eight cores and 8 threads and has a clock frequency of 3,2 GHz.

The Program:

- - The program will run:
 - Normal sequenFal matrix mulFplicaFon
 - Transposed matrix A sequenFal matrix mulFplicaFon
 - Transposed matrix B sequenFal matrix mulFplicaFon
 - Parallel normal matrix mulFplicaFon
 - Parallel Transposed matrix A matrix mulFplicaFon
 - Parallel Transposed matrix b matrix mulFplicaFon
- - For all of these runs, the program will use matrixes of varied sizes consisFng of 100x100, 200x200, 500x500 and 1000x1000.
- - To run the program, you must compile all the files and call upon main:
 - **javac *.java → java Main <Seed> <n>**
- - If you want to check and see if the matrixes are correct you run:
 - **Java Main <Seed> <n> test**

Implementa>on:

SequenFal:

- Normal: We create a new matrix called “m3” that contains the result of mulFplying every row in matrix1 “m1” with every column in matrix2 “m2”.

We do this using a triple for-loop which gives us $O(n^3)$

- Transposed A: We transpose A before we mulFply every column in A(m1) with every row in B(m2) and store and return the results in C (m3)
- Transposed B: We transpose B before we mulFply every column in A(m1) with every row in B(m2) and store and return the results in C (m3)

Parallel:

- Before we start parallelizing, we first find out how many cores we use. By

dividing N (length of the matrix sides) with available cores. If we have more cores than N , we create N amount of threads. This is because if we were to create more, it would be difficult to divide the work properly. If we have fewer cores than N , we create as many threads as we have cores and determine the remainders by dividing $N \% \text{cores}$ and give the excess workload to the first thread.

- We assign a secFon to each thread, where the threads take in the same empty matrix and fills in the results according to their given secFon.
- Transposing is the same in parallelizing.
- We create and store all the threads before we start them and waits for all to

finish using join()

Results:

	Seq nor	Seq A	Seq B	Para nor	Para A	Para B
100x100	1	1	1	1	3	1
200x200	11	11	10	3	8	3
500x500	191	404	179	36	131	25
1000x1000	2198	4167	1562	676	1550	201

Speed up

	Seq nor	Seq A	Seq B	Para nor	Para A	Para B
100x100	0%	0%	0%	0%	-67%	0%
200x200	0%	0%	10%	266,6%	37,5%	266,6%
500x500	0%	-53%	6%	430,5%	45%	664%
1000x1000	0%	-48%	40,7%	225,1%	41%	993,5%

Conclusion:

We can see very minor and not noteworthy improvements between the different sequenFal procedures. Maybe I have implemented it wrong, because some I have talked to got different answers, but we did get really big speedups when we compare the sequenFal with the parallelized procedures.

On the smaller matrixes there are some improvement, but the biggest improvements comes when the matrixes are large. Not in the case for Para A, even if it did have cachemisses, you would expect it to run faster because of more compuFng power than the sequenFal procedures, so I think the fault might lay in the code.

For Para B we can see that it performed really well in these test if we use the Seq Nor as the benchmark. This is might be because when transposing b and accessing the row by row, we have less cachemisses which greatly increased the speed.